

Project 1

name: 谢启强

student ID: 25303070189

github:

<https://github.com/meyHsieh/FDU/tree/e1d31d980043f8be66b490151ea2f5dec0ecdb05/DL/PJ1>

modelscope: <https://www.modelscope.cn/models/myHsieh/ML-project-1>

1. MLP

1.1 Linear Layer

- 输入矩阵: $\mathbf{X} \in \mathbb{R}^{\text{batch_size} \times \text{in_dim}}$
- 权重矩阵: $\mathbf{W} \in \mathbb{R}^{\text{in_dim} \times \text{out_dim}}$
- 偏置向量: $\mathbf{b} \in \mathbb{R}^{1 \times \text{out_dim}}$
- 输出矩阵: $\mathbf{Z} \in \mathbb{R}^{\text{batch_size} \times \text{out_dim}}$
- 损失函数: $L \in \mathbb{R}$ (标量)
- 网络结构: $\text{batch} \times 784 \xrightarrow{\text{Linear}(784, 600)} \text{batch} \times 600 \xrightarrow{\text{ReLU}} \text{batch} \times 600 \xrightarrow{\text{Linear}(600, 10)} \text{batch} \times 10 \xrightarrow{\text{Softmax}} \text{batch} \times 10 \xrightarrow{\text{CrossEntropy}} L \in \mathbb{R}$ (标量)

1.1.1 前向传播公式:

$$\mathbf{Z} = \mathbf{XW} + \mathbf{b} \implies Z_{ij} = \sum_{k=1}^{\text{in_dim}} X_{ik} W_{kj} + b_j$$

1.1.2 反向传播梯度推导

设损失函数为 L , 上游传回的梯度为 $\frac{\partial L}{\partial \mathbf{Z}} = \mathbf{G} \in \mathbb{R}^{\text{batch_size} \times \text{out_dim}}$ 。

1. 权重梯度 $\frac{\partial L}{\partial \mathbf{W}}$:

依据多变量链式法则:

$$\frac{\partial L}{\partial W_{uv}} = \sum_{i=1}^{\text{batch_size}} \sum_{j=1}^{\text{out_dim}} \frac{\partial L}{\partial Z_{ij}} \cdot \frac{\partial Z_{ij}}{\partial W_{uv}}$$

计算 $\frac{\partial Z_{ij}}{\partial W_{uv}}$:

$$Z_{ij} = \sum_{k=1}^{\text{in_dim}} X_{ik} W_{kj} + b_j$$

对 W_{uv} 求偏导, 仅当 $k = u$ 且 $j = v$ 时非零:

$$\frac{\partial Z_{ij}}{\partial W_{uv}} = X_{iu} \cdot \delta_{jv}$$

代回链式法则:

$$\frac{\partial L}{\partial W_{uv}} = \sum_{i=1}^{\text{batch_size}} \sum_{j=1}^{\text{out_dim}} G_{ij} \cdot X_{iu} \cdot \delta_{jv}$$

消去 δ_{jv} (仅 $j = v$ 的项保留) :

$$\frac{\partial L}{\partial W_{uv}} = \sum_{i=1}^{\text{batch_size}} X_{iu} G_{iv} = \sum_{i=1}^{\text{batch_size}} (\mathbf{X}^\top)_{ui} G_{iv}$$

矩阵形式:

$$\boxed{\frac{\partial L}{\partial \mathbf{W}} = \mathbf{X}^\top \mathbf{G}}$$

2. 输入梯度 $\frac{\partial L}{\partial \mathbf{X}}$:

$$\frac{\partial L}{\partial X_{uv}} = \sum_{i=1}^{\text{batch_size}} \sum_{j=1}^{\text{out_dim}} \frac{\partial L}{\partial Z_{ij}} \cdot \frac{\partial Z_{ij}}{\partial X_{uv}}$$

计算 $\frac{\partial Z_{ij}}{\partial X_{uv}}$:

$$Z_{ij} = \sum_{k=1}^{\text{in_dim}} X_{ik} W_{kj} + b_j$$

当 $i = u$ 时, Z_{uj} 中含 X_{uv} , 偏导为 W_{vj} ; 当 $i \neq u$ 时为零:

$$\frac{\partial Z_{ij}}{\partial X_{uv}} = \delta_{iu} \cdot W_{vj}$$

代回链式法则并消去 δ_{iu} :

$$\frac{\partial L}{\partial X_{uv}} = \sum_{j=1}^{\text{out_dim}} G_{uj} W_{vj} = \sum_{j=1}^{\text{out_dim}} G_{uj} (\mathbf{W}^\top)_{jv}$$

矩阵形式:

$$\boxed{\frac{\partial L}{\partial \mathbf{X}} = \mathbf{G} \mathbf{W}^\top}$$

3. 偏置梯度 $\frac{\partial L}{\partial \mathbf{b}}$:

$$\frac{\partial L}{\partial b_v} = \sum_{i=1}^{\text{batch_size}} \sum_{j=1}^{\text{out_dim}} \frac{\partial L}{\partial Z_{ij}} \cdot \frac{\partial Z_{ij}}{\partial b_v}$$

由于 $Z_{ij} = \sum_k X_{ik} W_{kj} + b_j$, $\frac{\partial Z_{ij}}{\partial b_v} = \delta_{jv}$:

$$\frac{\partial L}{\partial b_v} = \sum_{i=1}^{\text{batch_size}} \sum_{j=1}^{\text{out_dim}} G_{ij} \cdot \delta_{jv} = \sum_{i=1}^{\text{batch_size}} G_{iv}$$

矩阵形式:

$$\boxed{\frac{\partial L}{\partial \mathbf{b}} = \sum_{i=1}^{\text{batch_size}} \mathbf{G}_{i,:}}$$

1.1.3 He initialization

因为激活层为ReLU，加He初始化的原因是为了防止CNN模型的权重过大或过小。具体在CNN部分介绍。

```
In [ ]: class Linear(Layer):
    """
    The linear layer for a neural network. You need to implement the forward function and t
    """
    def __init__(self, in_dim, out_dim, initialize_method=np.random.normal, weight_decay=Fa
        super().__init__()
        scale = np.sqrt(2.0 / in_dim)
        self.W = initialize_method(loc=0.0, scale=scale, size=(in_dim, out_dim))
        self.b = np.zeros((1, out_dim))
        """
        self.W = initialize_method(size=(in_dim, out_dim))
        self.b = initialize_method(size=(1, out_dim))
        """
        self.grads = {'W' : None, 'b' : None}
        self.input = None # Record the input for backward process.

        self.params = {'W' : self.W, 'b' : self.b}

        self.weight_decay = weight_decay # whether using weight decay
        self.weight_decay_lambda = weight_decay_lambda # control the intensity of weight de

    def __call__(self, X) -> np.ndarray:
        return self.forward(X)

    def forward(self, X):
        self.input = X
        output = X @ self.params['W'] + self.params['b']
        return output

    def backward(self, grad : np.ndarray):
        self.grads['W'] = self.input.T @ grad
        self.grads['b'] = np.sum(grad, axis=0, keepdims=True)
        return grad @ self.params['W'].T

    def clear_grad(self):
        self.grads = {'W' : None, 'b' : None}
```

1.2 Multi-Cross-Entropy Loss (with Softmax)

- 输出矩阵 $\mathbf{Z} \in \mathbb{R}^{\text{batch_size} \times \text{max_classes}}$
- 标签 $\text{labels} \in \mathbb{N}^{\text{batch_size}}$
- 预测概率 $\mathbf{P} \in \mathbb{R}^{\text{batch_size} \times \text{max_classes}}$
- 损失函数 $L \in \mathbb{R}$

1.2.1 Forward

$$P_{ij} = \frac{e^{Z_{ij}}}{\sum_{k=1}^{\text{max_classes}} e^{Z_{ik}}}$$
$$L = -\frac{1}{\text{batch_size}} \sum_{i=1}^{\text{batch_size}} \ln P_{i, \text{labels}[i]}$$

1.2.2 Backward

推导中，使用 one-hot 形式 $\mathbf{Y}$ ($Y_{ij} = 1$ 当且仅当样本 i 的真实类别为 j)

链式法则：

$$\frac{\partial L_i}{\partial Z_{il}} = \sum_{j=1}^{\max_classes} \frac{\partial L_i}{\partial P_{ij}} \frac{\partial P_{ij}}{\partial Z_{il}}$$

损失函数对 Softmax 输出的梯度

$$L_i = - \sum_{j=1}^{\max_classes} Y_{ij} \ln P_{ij} = \ln P_{i, labels[i]}$$

对 P_{ij} 求偏导

$$\frac{\partial L_i}{\partial P_{ij}} = -Y_{ij} \cdot \frac{1}{P_{ij}} = \boxed{-\frac{Y_{ij}}{P_{ij}}}$$

Softmax 对 Logit 的梯度

$$P_{ij} = \frac{e^{Z_{ij}}}{\sum_{k=1}^{\max_classes} e^{Z_{ik}}} = \frac{e^{Z_{ij}}}{S_i}, \quad \text{其中 } S_i = \sum_{k=1}^{\max_classes} e^{Z_{ik}}$$

情况 1: $j = l$ (对同一个 logit 求导)

分子和分母都含 $e^{Z_{il}}$

$$\frac{\partial P_{il}}{\partial Z_{il}} = \frac{e^{Z_{il}} \cdot S_i - e^{Z_{il}} \cdot e^{Z_{il}}}{S_i^2}$$

分子提公因子 $e^{Z_{il}}$ ：

$$= \frac{e^{Z_{il}}(S_i - e^{Z_{il}})}{S_i^2} = \frac{e^{Z_{il}}}{S_i} \cdot \frac{S_i - e^{Z_{il}}}{S_i} = P_{il} \cdot (1 - P_{il})$$

情况 2: $j \neq l$ (对不同 logit 求导)

分子 $e^{Z_{ij}}$ 与 Z_{il} 无关，其导数为 0。只有分母的导数含 $e^{Z_{il}}$

$$\frac{\partial P_{ij}}{\partial Z_{il}} = \frac{0 \cdot S_i - e^{Z_{ij}} \cdot e^{Z_{il}}}{S_i^2} = -\frac{e^{Z_{ij}}}{S_i} \cdot \frac{e^{Z_{il}}}{S_i} = -P_{ij} \cdot P_{il}$$

统一形式：

两种情况可用 Kronecker delta δ_{jl} 统一为：

$$\boxed{\frac{\partial P_{ij}}{\partial Z_{il}} = P_{ij}(\delta_{jl} - P_{il})}, \quad \delta_{jl} = \begin{cases} 1 & j = l \\ 0 & j \neq l \end{cases}$$

组合：

$$\frac{\partial L_i}{\partial Z_{il}} = \sum_{j=1}^{\max_classes} \left(-\frac{Y_{ij}}{P_{ij}} \right) P_{ij}(\delta_{jl} - P_{il}) = - \sum_{j=1}^{\max_classes} Y_{ij}(\delta_{jl} - P_{il})$$

利用 one-hot 编码

$$-\left(\sum_{j=1}^{\max_classes} Y_{ij} \delta_{jl} - P_{il} \sum_{j=1}^{\max_classes} Y_{ij}\right) = -(Y_{il} - P_{il}) = P_{il} - Y_{il}$$

Batch 平均:

$$\frac{\partial L}{\partial Z_{il}} = \frac{1}{\text{batch_size}} (P_{il} - Y_{il})$$

$$\boxed{\frac{\partial L}{\partial \mathbf{Z}} = \frac{1}{\text{batch_size}} (\mathbf{P} - \mathbf{Y})}$$

```
In [ ]: class MultiCrossEntropyLoss(Layer):
    """
    A multi-cross-entropy loss layer, with Softmax layer in it, which could be cancelled by
    """
    def __init__(self, model = None, max_classes = 10) -> None:
        self.model = model
        self.max_classes = max_classes
        self.has_softmax = True
        self.predicts = None
        self.labels = None
        self.probs = None
        self.grads = None

    def __call__(self, predicts, labels):
        return self.forward(predicts, labels)

    def forward(self, predicts, labels):
        self.predicts = predicts
        self.labels = labels
        self.probs = softmax(predicts)

        correct_probs = self.probs[np.arange(len(labels)), labels]
        loss = -np.mean(np.log(correct_probs + 1e-10))
        return loss

    def backward(self):
        batch_size = len(self.labels)
        num_classes = self.probs.shape[1]

        self.grads = self.probs.copy()
        self.grads[np.arange(batch_size), self.labels] -= 1

        self.grads = self.grads / batch_size
        self.model.backward(self.grads)

    def cancel_soft_max(self):
        self.has_softmax = False
        return self
```

在交叉熵损失函数中加上小量: $loss = -np.mean(np.log(correct_probs + 1e-10))$, 否则可能超出浮点范围出现 $\log 0$ 的 inf

```
epoch: 3, iteration: 1300
[Train] loss: inf, score: 0.125
[Dev] loss: inf, score: 0.0756
epoch: 3, iteration: 1400
[Train] loss: inf, score: 0.09375
[Dev] loss: inf, score: 0.0756
```

1.3 Modification

*models.py*中原代码有误

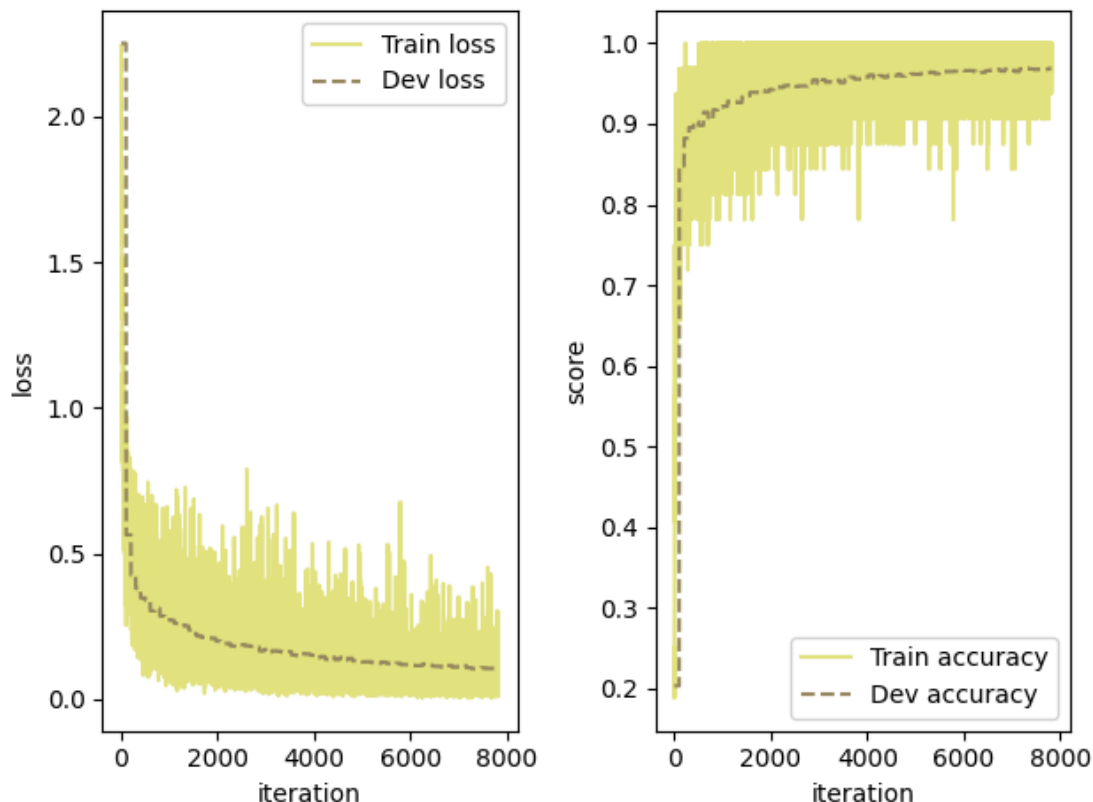
```
In [ ]: """for i in range(len(self.size_list) - 1):
        self.layers = []
        for i in range(len(self.size_list) - 1):
            layer = Linear(in_dim=self.size_list[i], out_dim=self.size_list[i + 1])
            layer.W = param_list[i + 2]['W']
            layer.b = param_list[i + 2]['b']
            layer.params['W'] = layer.W
            layer.params['b'] = layer.b
            layer.weight_decay = param_list[i + 2]['weight_decay']
            layer.weight_decay_lambda = param_list[i+2]['lambda']
            if self.act_func == 'Logistic':
                raise NotImplemented
            elif self.act_func == 'ReLU':
                layer_f = ReLU()
            self.layers.append(layer)
            if i < len(self.size_list) - 2:
                self.layers.append(layer_f)"""
for i in range(len(self.size_list) - 1):
    layer = Linear(in_dim=self.size_list[i], out_dim=self.size_list[i + 1])
    layer.W = param_list[i + 2]['W']
    layer.b = param_list[i + 2]['b']
    layer.params['W'] = layer.W
    layer.params['b'] = layer.b
    layer.weight_decay = param_list[i + 2]['weight_decay']
    layer.weight_decay_lambda = param_list[i+2]['lambda']
    if self.act_func == 'Logistic':
        raise NotImplemented
    elif self.act_func == 'ReLU':
        layer_f = ReLU()
    self.layers.append(layer)
    if i < len(self.size_list) - 2:
        self.layers.append(layer_f)
```

*MLP(baseline)*并不要求实现*learning rate scheduling*和*Regularization*, 注释修改相关代码:

```
In [ ]: """linear_model = nn.models.Model_MLP([train_imgs.shape[-1], 600, 10], 'ReLU', [1e-4, 1e-4])
linear_model = nn.models.Model_MLP([train_imgs.shape[-1], 600, 10], 'ReLU')
optimizer = nn.optimizer.SGD(init_lr=0.06, model=linear_model)
"""scheduler = nn.lr_scheduler.MultiStepLR(optimizer=optimizer, milestones=[800, 2400, 4000])
loss_fn = nn.op.MultiCrossEntropyLoss(model=linear_model, max_classes=train_labs.max()+1)
"""runner = nn.runner.RunnerM(linear_model, optimizer, nn.metric.accuracy, loss_fn, scheduler)
runner = nn.runner.RunnerM(linear_model, optimizer, nn.metric.accuracy, loss_fn)
```

1.4 training and validation performance

Setting	Best Dev Acc	Test Acc
MLP Baseline	0.9689	0.9718



2. CNN

2.1 Conv2D

- 输入: $\mathbf{X} \in \mathbb{R}^{\text{batch} \times \text{in_channels} \times H \times W}$
- 卷积核: $\mathbf{W} \in \mathbb{R}^{\text{out_channels} \times \text{in_channels} \times \text{kernel_size} \times \text{kernel_size}}$
- 偏置: $\mathbf{b} \in \mathbb{R}^{\text{out_channels} \times 1}$
- 输出: $\mathbf{Y} \in \mathbb{R}^{\text{batch} \times \text{out_channels} \times H_{\text{out}} \times W_{\text{out}}}$
- 步长: `self.stride = s`
- 损失函数: $L \in \mathbb{R}$ (标量)
- 网络结构: $\text{batch} \times 1 \times 28 \times 28 \xrightarrow{\text{conv2D}(1, 32, 5, S=1)} \text{batch} \times 32 \times 24 \times 24 \xrightarrow{\text{ReLU}} \text{batch} \times 32 \times 24 \times 24 \xrightarrow{\text{Flatten}} \text{batch} \times 18432 \xrightarrow{\text{Linear}(18432, 10)} \text{batch} \times 10 \xrightarrow{\text{Softmax}} \text{batch} \times 10 \xrightarrow{\text{CrossEntropy}} L \in \mathbb{R}$ (标量)

2.1.1 向前传播

输入是一批图像, 形状为 $N \times C_{\text{in}} \times H \times W$, 分别表示样本数、输入通道数、高度、宽度。卷积核的形状是 $C_{\text{out}} \times C_{\text{in}} \times k \times k$, 分别表示输出通道数、输入通道数、卷积核高度、卷积核宽度。

对于每个输出通道 c_{out} , 每个输出位置 (h, w) , 计算方式为: 取出输入中以 $(h \cdot s, w \cdot s)$ 为起点的 $k \times k$ 窗口, 和卷积核逐元素相乘再求和, 最后加上偏置。用公式表示就是:

$$\mathbf{Y}_{n, c_{\text{out}}, h, w} = \sum_{c_{\text{in}}=0}^{C_{\text{in}}-1} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} \mathbf{W}_{c_{\text{out}}, c_{\text{in}}, i, j} \cdot \mathbf{X}_{n, c_{\text{in}}, h \cdot s + i, w \cdot s + j} + \mathbf{b}_{c_{\text{out}}}$$

其中 s 是步长。输入是 28×28 , 卷积核是 5×5 , 步长为 1, 那么输出就是 24×24 。计算公式为:

$$H_{out} = \left\lfloor \frac{H - k}{s} \right\rfloor + 1, \quad W_{out} = \left\lfloor \frac{W - k}{s} \right\rfloor + 1$$

2.1.2 反向传播

基于 Im2Col 展开后的矩阵形式，前向传播可写为：

$$\mathbf{Y}_{col} = \mathbf{W}_{col} \cdot \mathbf{X}_{col}$$

其中：

- $\mathbf{X}_{col} \in \mathbb{R}^{(C_{in} \cdot k^2) \times (N \cdot H_{out} \cdot W_{out})}$ 为 Im2Col 展开后的输入矩阵
- $\mathbf{W}_{col} \in \mathbb{R}^{C_{out} \times (C_{in} \cdot k^2)}$ 为展开后的权重矩阵
- $\mathbf{Y}_{col} \in \mathbb{R}^{C_{out} \times (N \cdot H_{out} \cdot W_{out})}$ 为输出矩阵（展开形式）

反向传播时，上传传回的梯度为 $\frac{\partial L}{\partial \mathbf{Y}_{col}}$ ，形状与 $\mathbf{Y}_{col}$ 相同。需要计算三个梯度。

1. 权重梯度 $\frac{\partial L}{\partial \mathbf{W}_{col}}$

由矩阵乘法求导法则——对 $\mathbf{Y}_{col} = \mathbf{W}_{col} \cdot \mathbf{X}_{col}$ ，有

$$\frac{\partial L}{\partial \mathbf{W}_{col}} = \frac{\partial L}{\partial \mathbf{Y}_{col}} \cdot \mathbf{X}_{col}^\top$$

最后将 $\frac{\partial L}{\partial \mathbf{W}_{col}}$ reshape 回卷积核的形状 (C_{out}, C_{in}, k, k) 。

2. 偏置梯度 $\frac{\partial L}{\partial \mathbf{b}}$

偏置 $\mathbf{b} \in \mathbb{R}^{C_{out}}$ 对每个输出通道加一个标量。前向传播时 $Y_{n,c_{out},h,w}$ 中 $\mathbf{b}_{c_{out}}$ 的系数始终为 1，因此：

$$\frac{\partial L}{\partial \mathbf{b}_{c_{out}}} = \sum_{n=1}^N \sum_{h=1}^{H_{out}} \sum_{w=1}^{W_{out}} \frac{\partial L}{\partial Y_{n,c_{out},h,w}}$$

矩阵形式：

$$\frac{\partial L}{\partial \mathbf{b}} = \sum_{n,h,w} \frac{\partial L}{\partial \mathbf{Y}_{n,:,h,w}}$$

3. 输入梯度 $\frac{\partial L}{\partial \mathbf{X}}$

由矩阵乘法求导法则——对 $\mathbf{Y}_{col} = \mathbf{W}_{col} \cdot \mathbf{X}_{col}$ ，有

$$\frac{\partial L}{\partial \mathbf{X}_{col}} = \mathbf{W}_{col}^\top \cdot \frac{\partial L}{\partial \mathbf{Y}_{col}}$$

最后通过 col2im 将 $\frac{\partial L}{\partial \mathbf{X}_{col}}$ 从展开形式还原为原来的图像形状 (N, C_{in}, H, W) 。

2.1.3 Im2Col

卷积操作如果直接用四层循环实现（遍历样本、输出通道、输出高度、输出宽度），访存开销太大，在 Python 里会非常慢。

Im2Col 的是把卷积运算变成矩阵乘法。矩阵乘法是高度优化的运算，速度比 Python 循环快几十倍。

1.把输入图像展开成矩阵(以单batch为例)

以本实验为例，输入是一张 28×28 的单通道图像，卷积核是 5×5 ，步长为 1，输出是 24×24 。

输出大小是 24×24 ，所以输入图像上一共有 $24 \times 24 = 576$ 个窗口。每个窗口包含 $5 \times 5 = 25$ 个像素。

把这 576 个窗口每个都拉直成一个 25 维的列向量，横着排在一起，就得到矩阵 $\mathbf{X}_{col}$ ，大小是 25×576 。每一列是一个窗口的 25 个像素值。

2.把卷积核展开成矩阵

本实验有 32 个输出通道，每个通道的卷积核都是 $5 \times 5 = 25$ 个参数。把 32 个卷积核排成 32 行，每行 25 个数，得到矩阵 $\mathbf{W}_{col}$ ，大小是 32×25 。每一行就是一个输出通道的卷积核。

3.做矩阵乘法

原来的卷积运算现在变成了一个简单的矩阵乘法：

$$\mathbf{Y}_{col} = \mathbf{W}_{col} \cdot \mathbf{X}_{col}$$

$\mathbf{W}_{col}$ 是 32×25 ， $\mathbf{X}_{col}$ 是 25×576 ，乘出来的 $\mathbf{Y}_{col}$ 是 32×576 。每一行对应一个输出通道，每一列对应一个输出位置。

最后把 $\mathbf{Y}_{col}$ reshape 成 $(N, 32, 24, 24)$ 的形状，就得到了和直接用循环完全一样的结果。

4.反向传播也用矩阵乘法

反向传播时，输出梯度 $\frac{\partial L}{\partial \mathbf{Y}}$ 的形状是 $(N, 32, 24, 24)$ ，先 reshape 成 32×576 的矩阵。然后用 $\mathbf{W}_{col}$ 的转置 (25×32) 去乘，得到 25×576 的矩阵，这就是输入梯度的展开形式：

$$\frac{\partial L}{\partial \mathbf{X}_{col}} = \mathbf{W}_{col}^T \cdot \frac{\partial L}{\partial \mathbf{Y}_{col}}$$

最后通过 col2im 操作，把这个 25×576 的矩阵还原成 $(N, 1, 28, 28)$ 的梯度。

2.1.4 He 初始化

一开始我用均值为 0、标准差为 1.0 的正态分布随机初始化所有权重。输入图像的像素值是 0 到 1 之间（归一化后）。第一层卷积有 1 个输入通道，卷积核大小是 5×5 ，所以有 25 个连接。每个权重约为 1.0。当 25 个输入（约 0~1）和 25 个权重（约 1.0）相乘再相加时，结果的标准差大约是 $\sqrt{25} = 5$ 。经过 ReLU 激活函数后，负值被截断为 0，正值保留，标准差约为 2.5

到了下一层（Flatten 后的全连接层），输入变成了 $32 \times 24 \times 24 = 18432$ 个，每个权重 2.5。输出标准差大约是 $\sqrt{18432 \times 2.5} \approx 215$ 。经过 ReLU 后，标准差为 112.5。恰好 softmax 对方差较大的数据十分敏感， e^0 和 $e^{112.5}$ 相差极大，而且 $\frac{\partial L}{\partial \mathbf{Z}} = \frac{1}{\text{batch_size}} (\mathbf{P} - \mathbf{Y})$ 会让每次梯度下降的值都很小，模型难以收敛

He 初始化：对于有 D 个输入连接的层（卷积层中 $D = C_{in} \times k \times k$ ），把权重采样自均值为 0、标准差为 $\sqrt{2/D}$ 的正态分布：

$$\mathbf{W} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{C_{in} \cdot k^2}}\right)$$

这样就能保证全连接层的输出方差为 1，模型更容易收敛

```
In [ ]: class conv2D(Layer):
    def __init__(self, in_channels, out_channels, kernel_size, stride=1, padding=0, weight_
        super().__init__()
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.kernel_size = k = kernel_size
```

```

self.stride = stride
self.padding = padding

n_in = in_channels * kernel_size * kernel_size
scale = np.sqrt(2.0 / n_in)
self.W = np.random.normal(loc=0.0, scale=scale, size=(out_channels, in_channels, kernel_size, kernel_size))
self.b = np.zeros((out_channels, 1))

self.params = {'W': self.W, 'b': self.b}
self.grads = {'W': None, 'b': None}
self.weight_decay = weight_decay
self.weight_decay_lambda = weight_decay_lambda

def forward(self, X):
    self.input = X
    batch, C, H, W = X.shape
    k = self.kernel_size
    out_c = self.out_channels

    out_h = (H - k) // self.stride + 1
    out_w = (W - k) // self.stride + 1

    i0 = np.repeat(np.arange(k), k)
    i0 = np.tile(i0, C)
    i1 = self.stride * np.repeat(np.arange(out_h), out_w)
    j1 = self.stride * np.tile(np.arange(out_w), out_h)
    i = i0.reshape(-1, 1) + i1.reshape(1, -1)
    j = np.tile(np.arange(k), k * C).reshape(-1, 1) + j1.reshape(1, -1)
    k_idx = np.repeat(np.arange(C), k * k).reshape(-1, 1)

    self.X_col = X[:, k_idx, i, j].transpose(1, 2, 0).reshape(C * k * k, -1)

    W_col = self.params['W'].reshape(out_c, -1)
    out = W_col @ self.X_col + self.params['b']

    out = out.reshape(out_c, out_h, out_w, batch).transpose(3, 0, 1, 2)
    return out

def backward(self, grad):
    batch, out_c, out_h, out_w = grad.shape
    C, k = self.in_channels, self.kernel_size

    grad_col = grad.transpose(1, 2, 3, 0).reshape(out_c, -1)

    self.grads['W'] = (grad_col @ self.X_col.T).reshape(out_c, C, k, k)
    self.grads['b'] = np.sum(grad_col, axis=1, keepdims=True)

    W_col = self.params['W'].reshape(out_c, -1)
    dX_col = W_col.T @ grad_col

    dX = np.zeros_like(self.input)

    i0 = np.repeat(np.arange(k), k)
    i0 = np.tile(i0, C)
    i1 = self.stride * np.repeat(np.arange(out_h), out_w)
    j1 = self.stride * np.tile(np.arange(out_w), out_h)
    i = i0.reshape(-1, 1) + i1.reshape(1, -1)
    j = np.tile(np.arange(k), k * C).reshape(-1, 1) + j1.reshape(1, -1)
    k_idx = np.repeat(np.arange(C), k * k).reshape(-1, 1)

    k_idx_long = np.broadcast_to(k_idx, i.shape).ravel()
    i_long = i.ravel()
    j_long = j.ravel()

    dX_resaped = dX_col.reshape(C*k*k, out_h*out_w, batch).transpose(2, 0, 1)

    for b in range(batch):

```

```

        np.add.at(dX[b], (k_idx_long, i_long, j_long), dX_reshaped[b].ravel())

    return dX

def clear_grad(self):
    self.grads = {'W': None, 'b': None}

```

2.1.5 Flatten

```

In [ ]: class Flatten(Layer):
    def __init__(self) -> None:
        super().__init__()
        self.optimizable = False
        self.input_shape = None

    def __call__(self, X):
        return self.forward(X)

    def forward(self, X):
        self.input_shape = X.shape
        batch_size = X.shape[0]
        return X.reshape(batch_size, -1)

    def backward(self, grad):
        return grad.reshape(self.input_shape)

```

2.2 CNN model

```

In [ ]: class Model_CNN(Layer):
    def __init__(self):
        super().__init__()
        self.layers = []

        self.layers.append(conv2D(in_channels=1, out_channels=32, kernel_size=5, stride=1))
        self.layers.append(ReLU())
        self.layers.append(Flatten())
        self.layers.append(Linear(in_dim=18432, out_dim=10))

    def __call__(self, X):
        return self.forward(X)

    def forward(self, X):
        for layer in self.layers:
            X = layer.forward(X)
        return X

    def backward(self, loss_grad):
        for layer in reversed(self.layers):
            loss_grad = layer.backward(loss_grad)
        return loss_grad

    def save_model(self, save_path):
        param_list = []
        for layer in self.layers:
            if layer.optimizable:
                p = {
                    'W': layer.params['W'],
                    'b': layer.params['b'],
                    'weight_decay': layer.weight_decay,
                    'lambda': layer.weight_decay_lambda
                }
                param_list.append(p)

        with open(save_path, 'wb') as f:

```

```

        pickle.dump(param_list, f)

    def load_model(self, model_path):
        with open(model_path, 'rb') as f:
            param_list = pickle.load(f)

        optimizable_idx = 0
        for layer in self.layers:
            if layer.optimizable:
                layer.params['W'] = param_list[optimizable_idx]['W']
                layer.params['b'] = param_list[optimizable_idx]['b']
                layer.weight_decay = param_list[optimizable_idx]['weight_decay']
                layer.weight_decay_lambda = param_list[optimizable_idx]['lambda']
                optimizable_idx += 1

```

2.3 Modification

runner.py 进行一些调整，否则因为CNN参数太多，每次都去验证集上验证一下，训练会很慢

```

In [ ]: """
dev_score, dev_loss = self.evaluate(dev_set)
self.dev_scores.append(dev_score)
self.dev_loss.append(dev_loss)
"""

if iteration % log_iters == 0:
    dev_score, dev_loss = self.evaluate(dev_set)
    self.dev_scores.append(dev_score)
    self.dev_loss.append(dev_loss)
else:
    self.dev_scores.append(self.dev_scores[-1] if self.dev_scores else 0)
    self.dev_loss.append(self.dev_loss[-1] if self.dev_loss else 0)

```

2.4 compare the CNN with the MLP

比较条件: lr=0.06, 无学习率调度、正则化.....

- MLP网络结构: batch×784 —Linear(784,600)—► batch×600 —ReLU—► batch×600 —Linear(600,10)—► batch×10 —Softmax—► batch×10 —CrossEntropy—► $L \in \mathbb{R}$ (标量)
- CNN网络结构: batch×1×28×28 —conv2D(1,32,5,S=1)—► batch×32×24×24 —ReLU—► batch×32×24×24 —Flatten—► batch×18432 —Linear(18432,10)—► batch×10 —Softmax—► batch×10 —CrossEntropy—► $L \in \mathbb{R}$ (标量)

CNN 在 MNIST 上的表现,无论是验证集收敛速度还是测试集上的波动范围,都显著优于 MLP。

Why is CNN more suitable than MLP for image classification? Does the CNN improve validation or test accuracy?

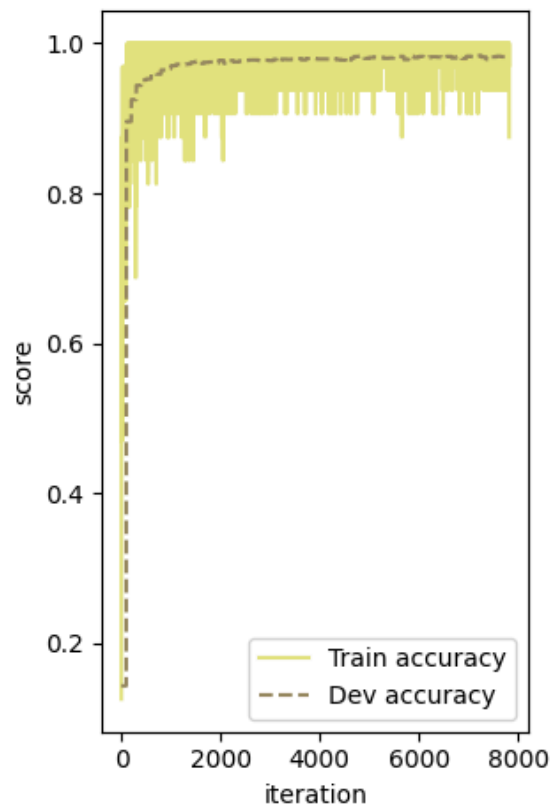
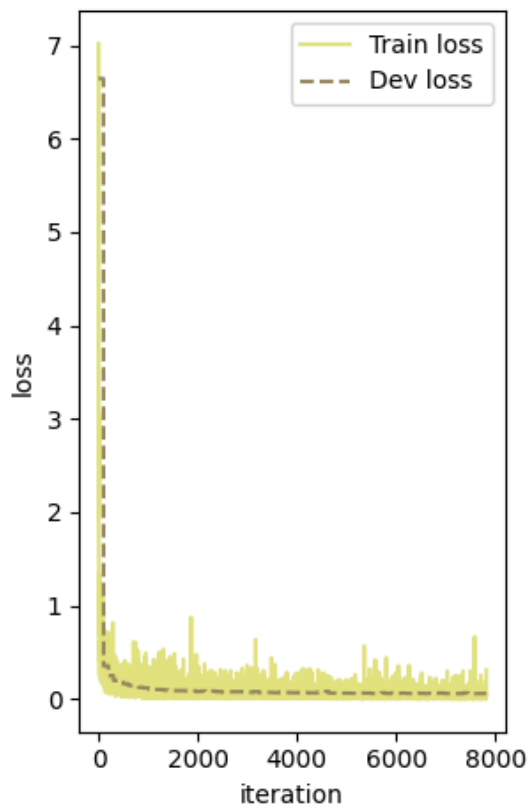
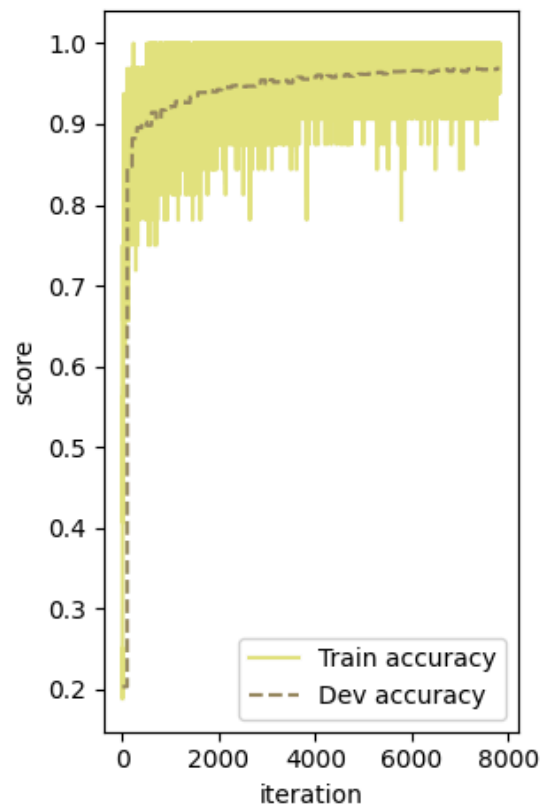
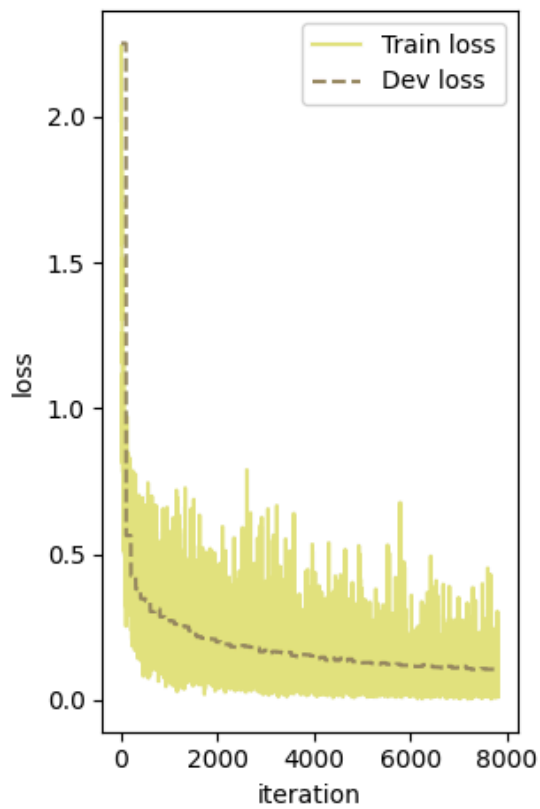
MLP 将 28×28 的图像拉直为 784 维向量,每个隐藏层神经元与全部 784 个输入相连。破坏了像素的空间关系。图像中相邻的像素在拉直后的向量中可能相距很远。例如,第一行最后一个像素与第二行第一个像素在图像中是上下相邻的,但在向量中相隔 28 个位置。

第二,参数量过大。本实验中 MLP 的第一层有 $784 \times 600 = 470400$ 个参数,总计约 47 万个参数。过多的参数不仅增加计算开销,还容易导致过拟合,模型可能记住训练集中的噪声,而非学习通用的特征。

CNN 采用 5×5 的卷积核在图像上滑动,每次仅处理局部区域。保留空间关系。卷积核内的 25 个像素在图像中是真实相邻的,CNN 能够自然地捕捉局部特征,如边缘、纹理和拐角。

第二,减少参数量。每个卷积核仅有 25 个参数,且同一卷积核在图像所有位置共享。

Setting	Best Dev Acc	Test Acc
MLP Baseline	0.9689	0.9718
CNN	0.9811	0.9837



3 Two Additional Directions

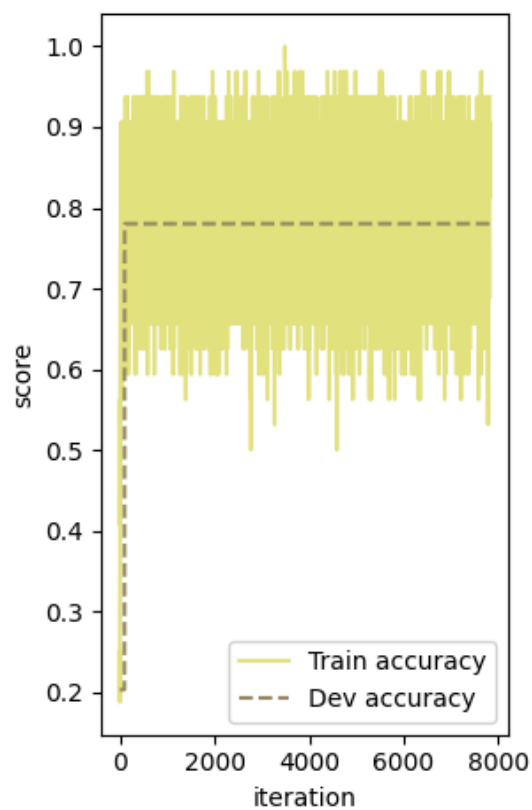
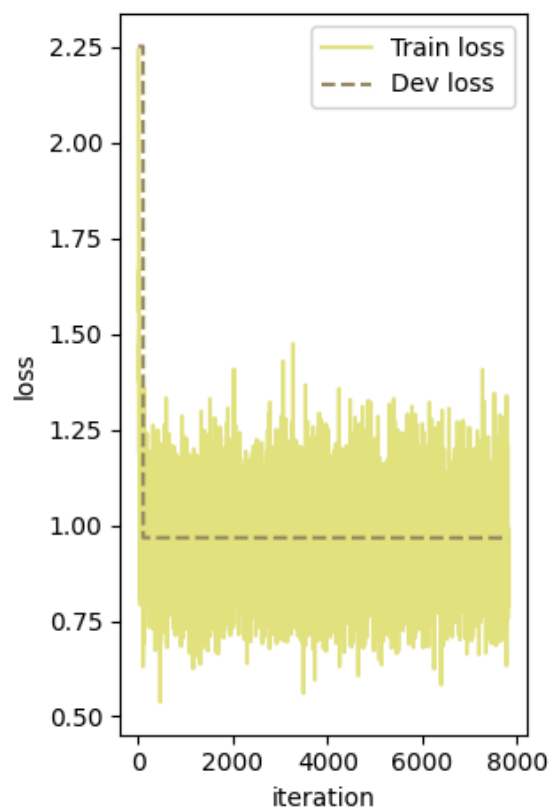
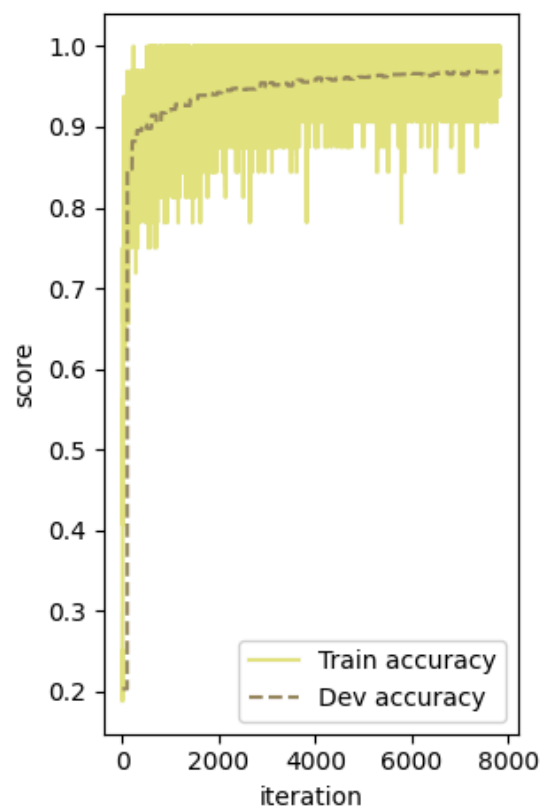
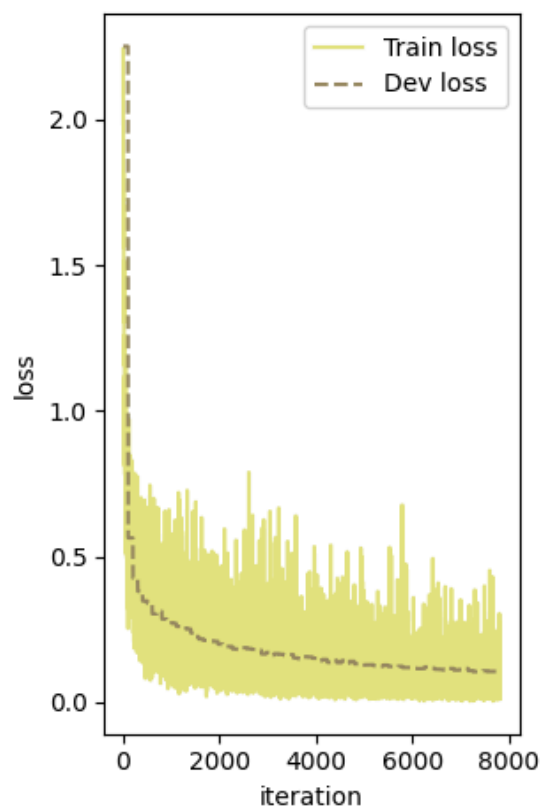
3.1 Optimization: learning rate scheduling

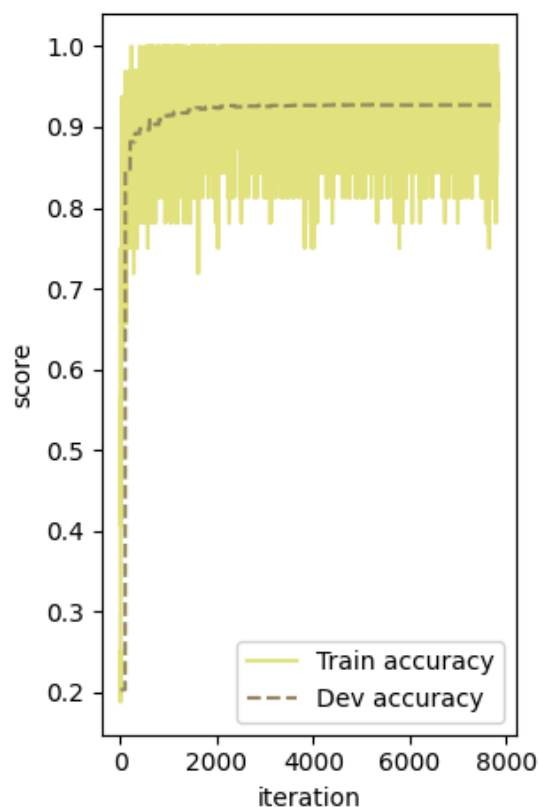
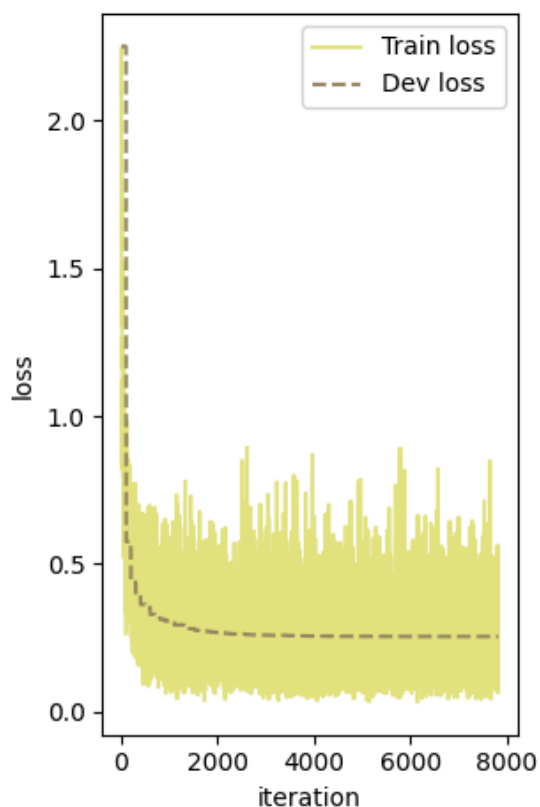
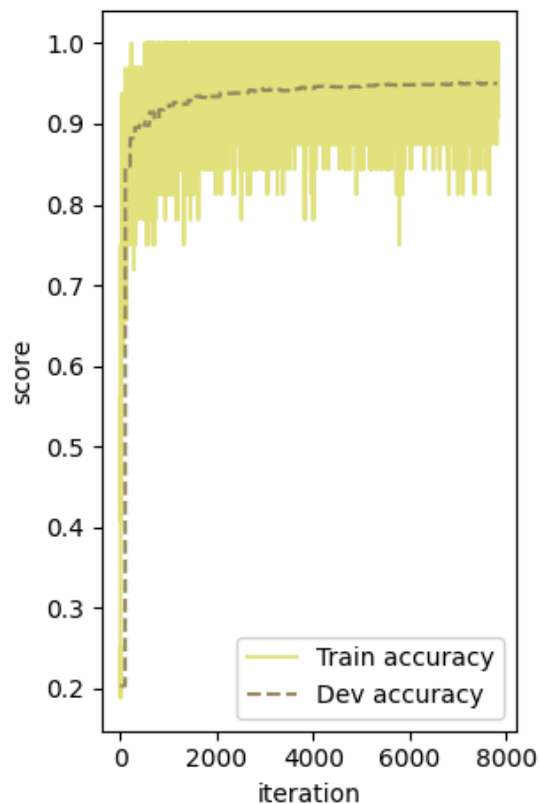
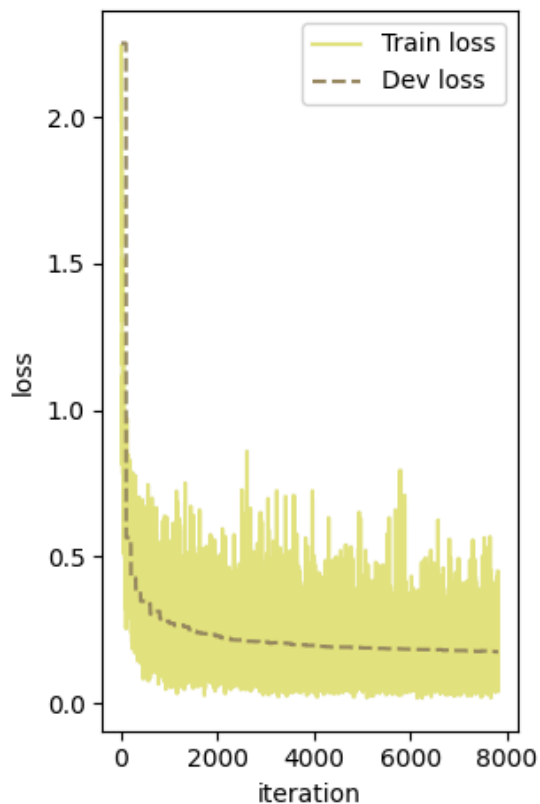
```
In [ ]: class MultiStepLR(scheduler):
    def __init__(self, optimizer, milestones, gamma=0.1) -> None:
        super().__init__(optimizer)
        self.milestones = set(milestones)
        self.gamma = gamma

    def step(self) -> None:
        self.step_count += 1
        if self.step_count in self.milestones:
            self.optimizer.init_lr *= self.gamma
            print(f"MultiStepLR: Learning rate reduced to {self.optimizer.init_lr} at step {self.step_count}")

class ExponentialLR(scheduler):
    def __init__(self, optimizer, gamma=0.999) -> None:
        super().__init__(optimizer)
        self.gamma = gamma

    def step(self) -> None:
        self.step_count += 1
        self.optimizer.init_lr *= self.gamma
```





Learning Rate Strategy	Best Dev Acc	Test Acc
MLP Baseline	0.9689	0.9718
StepLR	0.7802	0.7919
MultiStepLR	0.9501	0.9521
ExponentialLR	0.9274	0.9344

在 MLP Baseline (lr=0.06) 的基础上，分别测试了三种调度器：

- **StepLR**: 每 30 步将学习率衰减为原来的 0.1 倍。整个训练过程中学习率会衰减 260 多次
- **MultiStepLR**: 在 milestones=[800, 2400, 4000] 处将学习率衰减为原来的 0.5 倍。总共只衰减 3 次，最终学习率为 $0.06 \times 0.5^3 = 0.0075$
- **ExponentialLR**: 每步将学习率衰减为原来的 0.999 倍。训练结束时学习率为 $0.06 \times 0.999^{8000} \approx 0.06 \times 3.35 \times 10^{-4} \approx 2 \times 10^{-5}$ ，实际上也很小

从结果来看，StepLR 的表现最差，准确率只有 78% 左右。原因是 decay 频率太高，每 30 步就衰减一次，导致学习率在第一轮 epoch 之内就降到接近零。模型还没来得及学到什么东西，参数更新就停止了。从学习曲线上看，StepLR 的 loss 在训练初期短暂下降后就基本不再变化，模型根本学不动。

MultiStepLR 的表现相对接近 Baseline。它在 iteration=800 时第一次将学习率降至 0.03，此时模型已经完成了约 0.5 个 epoch 的训练，损失已大致收敛。第二次和第三次衰减进一步减小学习率，帮助模型在局部进行微调。但由于总共只训练了 5 个 epoch，衰减后的学习率 (0.0075) 虽然能继续微调，但时间不够充分。

ExponentialLR 的下降趋势与 StepLR 类似，但情况稍微好一点。不过由于衰减系数 0.999 导致训练后期学习率不到 10^{-4} ，模型同样在后期基本无法更新。

Lr对模型收敛效果的影响？

对于 MLP + MNIST，模型并不复杂，5 个 epoch 的训练过程中 loss 一直在稳定下降，在这种情况下，保持初始学习率 0.06 让模型在每个 epoch 都能以较大的步长更新参数，反而比提前衰减学习率更有利于快速收敛。也就是说，学习率调度的优势（在 loss 进入平台期后减小学习率进行精细调优）在只有 5 个 epoch 的训练中还没来得及体现。如果训练更多 epoch（比如 10~20 个），Baseline 可能会因为学习率太大而在最优解附近震荡，此时 MultiStepLR 这类调度策略的优势就会显现出来。

3.2 Error Analysis and Visualization

3.2.1 confusion matrix

```
In [ ]: # weight_visualization.py

import mynn as nn
import numpy as np
from struct import unpack
import gzip
import matplotlib.pyplot as plt
import pickle

model = nn.models.Model_MLP()
model.load_model(r'.\saved_models\MLP_baseline.pickle')

test_images_path = r'.\dataset\MNIST\t10k-images-idx3-ubyte.gz'
test_labels_path = r'.\dataset\MNIST\t10k-labels-idx1-ubyte.gz'

with gzip.open(test_images_path, 'rb') as f:
    magic, num, rows, cols = unpack('>4I', f.read(16))
    test_imgs = np.frombuffer(f.read(), dtype=np.uint8).reshape(num, 28*28)

with gzip.open(test_labels_path, 'rb') as f:
    magic, num = unpack('>2I', f.read(8))
    test_labs = np.frombuffer(f.read(), dtype=np.uint8)

test_imgs = test_imgs / test_imgs.max()

logits = model(test_imgs)
```

```

preds = np.argmax(logits, axis=-1)

print(f"Test Accuracy: {nn.metric.accuracy(logits, test_labs):.4f}")

#confusion matrix
def plot_confusion_matrix(y_true, y_pred, num_classes=10):
    cm = np.zeros((num_classes, num_classes), dtype=int)

    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1

    fig, ax = plt.subplots(figsize=(8, 6))
    im = ax.imshow(cm, cmap='Blues')

    for i in range(num_classes):
        for j in range(num_classes):
            text = ax.text(j, i, cm[i, j], ha="center", va="center",
                           color="white" if cm[i, j] > cm.max()/2 else "black",
                           fontsize=10)

    ax.set_xlabel('Predicted Label', fontsize=12)
    ax.set_ylabel('True Label', fontsize=12)
    ax.set_title('Confusion Matrix (MLP Baseline)', fontsize=14)
    ax.set_xticks(np.arange(num_classes))
    ax.set_yticks(np.arange(num_classes))
    ax.set_xticklabels([str(i) for i in range(num_classes)])
    ax.set_yticklabels([str(i) for i in range(num_classes)])

    plt.colorbar(im, ax=ax)
    plt.tight_layout()
    plt.savefig(r'..\Figs\MLP_Figure\confusion_matrix.png', dpi=150)
    plt.show()

    return cm

print("\nConfusion Matrix")
cm = plot_confusion_matrix(test_labs, preds)

for i in range(10):
    total = cm[i].sum()
    correct = cm[i, i]
    error_rate = (total - correct) / total * 100
    print(f"Digit {i}: {correct}/{total} correct, Error rate: {error_rate:.2f}%")

#Misclassified Examples
def plot_misclassified_examples(images, labels, preds, num_examples=20):
    cm = np.zeros((10, 10), dtype=int)
    for t, p in zip(labels, preds):
        cm[t, p] += 1

    worst = max(range(10), key=lambda i: 1 - cm[i,i]/cm[i].sum() if cm[i].sum() > 0 else 0)

    cm[worst, worst] = 0
    confused = np.argmax(cm[worst])

    idx = np.where((labels == worst) & (preds == confused))[0]
    selected = np.random.choice(idx, min(num_examples, len(idx)), replace=False)

    rows = (len(selected) + 4) // 5
    fig, axes = plt.subplots(rows, 5, figsize=(12, 2.5*rows))
    for i, ax in enumerate(axes.flat):
        if i < len(selected):
            ax.imshow(images[selected[i]].reshape(28,28), cmap='gray')
            ax.set_title(f'{worst}->{confused}')
            ax.axis('off')

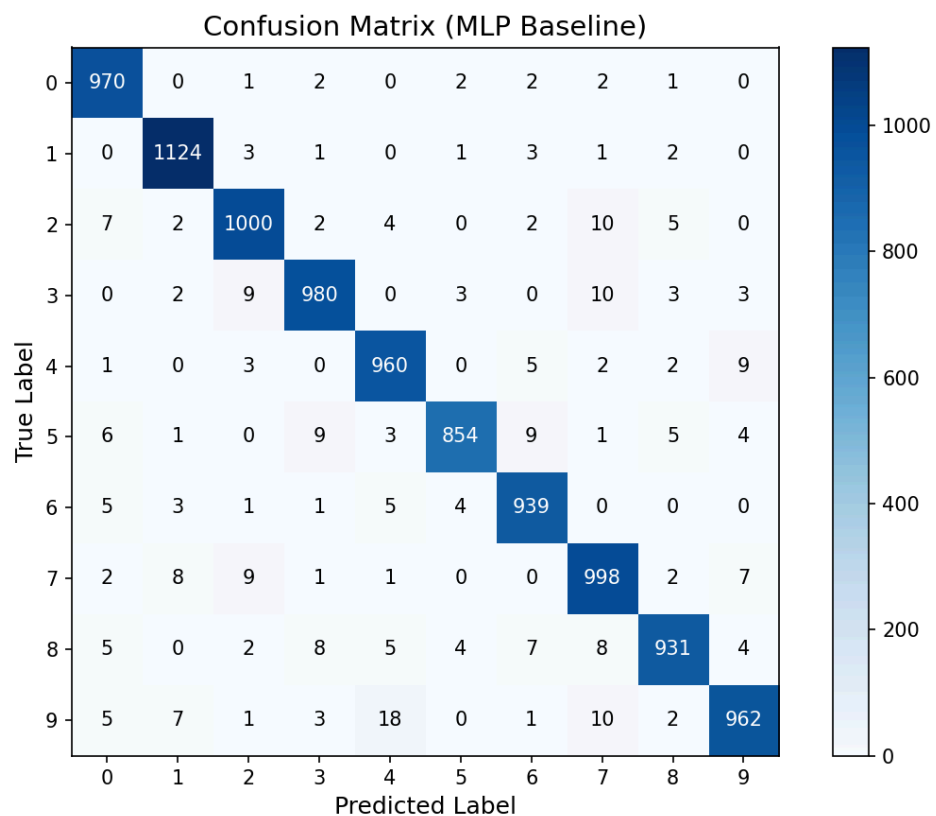
    plt.tight_layout()

```

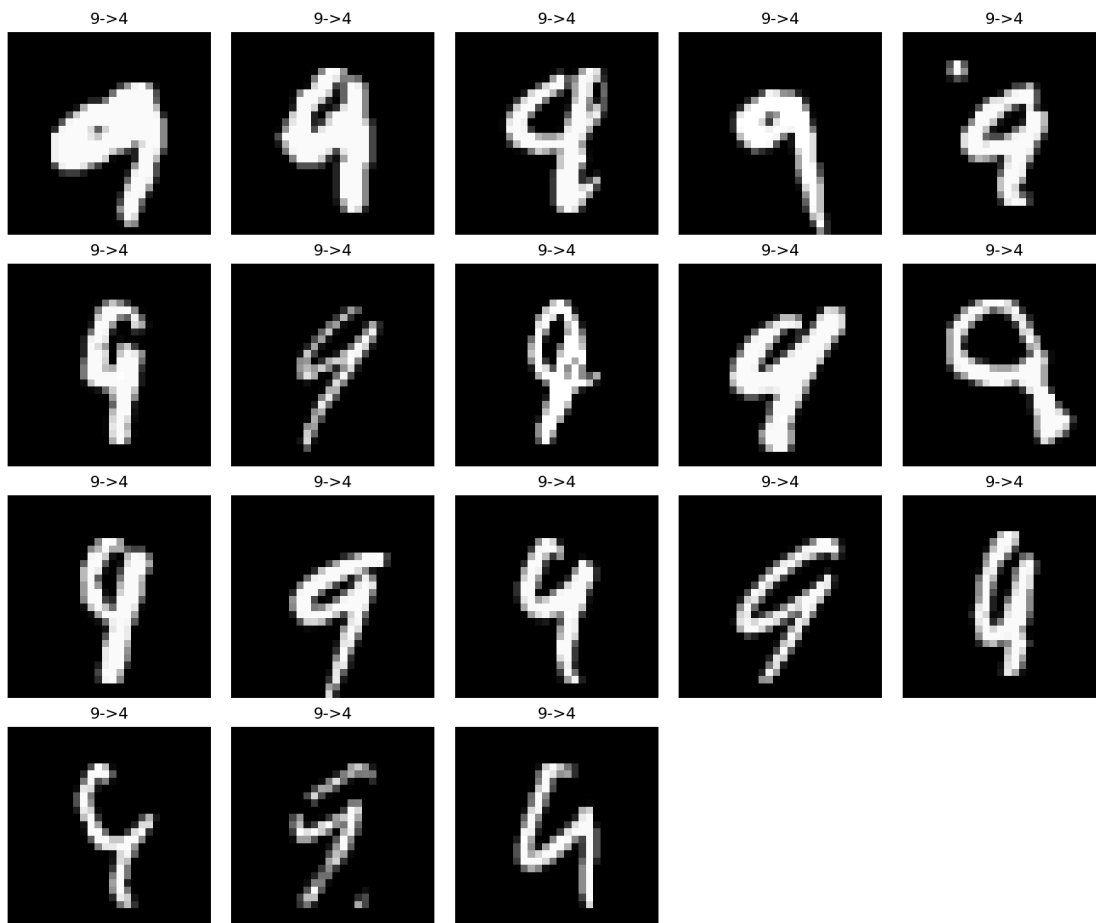
```
plt.savefig(r'..\Figs\MLP_Figure\misclassified.png', dpi=150)
plt.show()

print("\nMisclassified Examples")
plot_misclassified_examples(test_imgs, test_labs, preds, num_examples=20)
```

Digit	Correct / Total	Error Rate
0	970 / 980	1.02%
1	1124 / 1135	0.97%
2	1000 / 1032	3.10%
3	980 / 1010	2.97%
4	960 / 982	2.24%
5	854 / 892	4.26%
6	939 / 958	1.98%
7	998 / 1028	2.92%
8	931 / 974	4.41%
9	962 / 1009	4.66%



3.2.2 misclassified examples



本部分的特意展示了 baseline 模型识别错误率最高类别中的最容易被分错的类别展示

从混淆矩阵的对角线外元素来看，最常见的错误是 2、3 被错认成 7；9 被错认成 4 或 7 最多。9 的上半部分是一个圈，下面带一竖。如果圈写得不够圆，或者竖线偏左，看起来就像 4，如果竖线写得很直很长，又可能像 7。

从每个数字的错误率来看： 1.错误率最高的数字是 5（4.26%）、8（4.41%）、9（4.66%）。这些数字的结构都比较复杂，包含曲线、横线、竖线，不同人的写法差异很大。

2.错误率最低的数字是 0（1.02%）和 1（0.97%）。结构最简单（一个圈、一条竖线），变形空间小。

可能的原因：

1.某些数字的局部特征太像了。模型通过检测边缘、曲线等局部特征来分类。当两个数字共享某些特征时，模型就难以区分。比如“右上开口加弧线”的特征同时出现在 9 和 4 上(手写的)。

2.部分图片质量差。有些样本边缘模糊，特征不明显。